

Good Development Practice

There are any resources out there; this list not exhaustive

- "Good enough practices in scientific computing"
- "Best practices in scientific computing"
- Test Driven Development, credited to Kent Beck

Why this talk on good practice?

- From “Good enough practices in scientific computing:”
- Most researchers are never taught the equivalent of basic lab skills for research computing.
- As a result, data can get lost, analyses can take much longer than necessary, and researchers are limited in how effectively they can work with software and data.
- Computing workflows need to follow the same practices as lab projects and notebooks, with organized data, documented steps, and the project structured for reproducibility, but researchers new to computing often don’t know where to start.

From “Best practices”

- We believe that software is just another kind of experimental apparatus and should be built, checked, and used as carefully as any physical apparatus. However, while most scientists are careful to validate their laboratory and field equipment, most do not know how reliable their software is

"Good enough practices in scientific computing"

Outline (directly from the paper)

- Data management
- Software *
- Collaboration
- Project Organization *
- Keeping track of changes
- Manuscripts
- Items they don't mention

Data management

- Save the raw data.
- Ensure that raw data are backed up in more than one location.
- Create the data you wish to see in the world.
- Create analysis-friendly data.
- Record all the steps used to process data.
- Anticipate the need to use multiple tables, and use a unique identifier for every record.
- Submit data to a reputable DOI-issuing repository so that others can access and cite it.

Software

- Place a brief explanatory comment at the start of every program.
- Decompose programs into functions.
- Be ruthless about eliminating duplication.
- Always search for well-maintained software libraries that do what you need.
- Test libraries before relying on them.
- Give functions and variables meaningful names.
- Make dependencies and requirements explicit.
- Do not comment and uncomment sections of code to control a program's behavior.
- Provide a simple example or test data set.
- Submit code to a reputable DOI-issuing repository.

More on Software practices: libraries

- Python: SciPy, NumPy,...
- C++: STL, Boost
 - For example, there is a vector dot product in the C++ STL!
 - Library code runs better and faster than what you will write, unless you know you are writing a special case of a general algorithm (e.g. $x*x$ is faster than `pow(x,2)`)

Collaboration

- Create an overview of your project.
- Create a shared "to-do" list for the project.
- Decide on communication strategies.
- Make the license explicit.
- Make the project citable.

More on licenses

- Lack of an explicit license does not mean there isn't one; rather, it implies the author is keeping all rights and others are not allowed to reuse or modify the material.

Project Organization

- Put each project in its own directory, which is named after the project.
- Put text documents associated with the project in the doc directory.
- Put raw data and metadata in a data directory and files generated during cleanup and analysis in a results directory.
- Put project source code in the src directory, both core and `driver' scripts.
- Put external scripts or compiled programs in the bin directory.
- Name all files to reflect their content or function.

More on organization: sample project folder

```
| --CITATION
| --README
| --LICENSE
|
| --requirements.txt
| --data
|   |--birds_count_table.csv
|
| --doc
|   |--notebook.md
|   |--manuscript.md
|   |--changelog.txt
|
| --results
|   |--summarized_results.csv
|
| --src
|   |--sightings_analysis.py
|   |--runall.py
```

Keeping track of changes

- Backup (almost) everything created by a human being as soon as it is created.
- Keep changes small.
- Share changes frequently.
- Create, maintain, and use a checklist for saving and sharing changes to the project.
- Store each project in a folder that is mirrored off the researcher's working machine.
- Add a file called CHANGELOG.txt to the project's docs subfolder.
- Copy the entire project whenever a significant change has been made.
- Use a version control system, e.g. GitHub, BitBucket

Manuscripts

- An old joke says that doing the research is the first 90% of any project; writing up is the other 90%. While writing is rarely addressed in discussions of scientific computing, computing has changed scientific writing just as much as it has changed research.
- The workflow you choose is less important than having all authors agree on the workflow before writing starts. Make sure to also agree on a single method to provide feedback, be it an email thread or mailing list, an issue tracker (like the ones provided by GitHub and Bit- bucket), or some sort of shared online to-do list.
- Write manuscripts using online tools with rich formatting, change tracking, and reference management.
- Write the manuscript in a plain text format that permits version control.

Goals of manuscript workflow decisions

- Ensure that text is accessible to yourself and others now and in the future by making a single master document that is available to all coauthors at all times.
- Reduce the chances of work being lost or people overwriting each other's work.
- Make it easy to track and combine contributions from multiple collaborators.
- Avoid duplication and manual entry of information, particularly in constructing bibliographies, tables of contents, and lists.
- Make it easy to regenerate the final published form (e.g., a PDF) and to tell if it is up to date.
- Make it easy to share that final version with collaborators and to submit it to a journal.

What they didn't say

- More advanced concepts, maybe not worth the investment for small projects
 - Branches
 - Build tools
 - Unit tests
 - Coverage
 - Continuous Integration
 - Profiling and optimization
 - Semantic web
 - Extended documentation
 - Code reviews/ pair programming
- Beyond the scope
 - Bibliography manager

Useful points from “Best Practices”

- scientists often can't know what their programs should do next until the current version has produced some results.
- Work in steps of about an hour, projects up to a week, aiming for working code at the end of each iteration
- Known as “agile development”

Test Driven Development

TDD, or Red/Green/Refactor

- Write a test to capture a desired functionality
- Make sure the test fails (and previous tests pass)
- Write the code
- Run (and pass) tests
- Refactor – move new code into logically correct place and make sure code is clean and readable
- Repeat

Other good practices

- Use an Integrating Developing Environment (IDE)
 - On Macs: Xcode
 - On Windows: Visual Studio
 - Eclipse, Netbeans,...
- Clion, Visual Studio Code
- The investment pays off! It simplifies debugging, linking, building, editing, formatting, and more

Other Resources

- Software Carpentry (much of “Good practices” from this group)
- <https://software-carpentry.org/blog/2017/06/good-enough.html>

For next time

- Best practices for profiling and optimization?
- How to reconcile agile adaptive programming (small changes, frequent feedback, working code all along) vs. TDD? How to do scientific TDD?
- Two types of code – low level functions and high level scientific code? Try to do as little of the latter (and hence few automated tests) as possible. “Work in as high a level language as possible” – MATLAB and .mex files is a good compromise
- What are coverage, continuous integration?
- Other testing paradigms – smoke testing, continuous testing, unit test packages. Automation of tests is part of the planning process, not post-coding, see <https://www.forbes.com/sites/forbestechcouncil/2017/10/30/how-to-pass-the-software-development-test/#d8c1bf82f019>
- Writing good unit tests: more about code design than good tests; write modular code that performs simple functions on simple pieces of data, see <https://www.toptal.com/qa/how-to-write-testable-code-and-why-it-matters>
- Four kinds of testing, see <https://www.process.st/types-of-software-testing/>
- Unit tests, integration tests: user needs to see integration test results, not unit tests, system tests, beta or black body tests

“Best practices in scientific computing”

Summary of Best Practices

1. Write programs for people, not computers.
2. Let the computer do the work.
3. Make incremental changes.
4. Don't repeat yourself (or others).
5. Plan for mistakes.
6. Optimize software only after it works correctly.
7. Document design and purpose, not mechanics.
8. Collaborate.

Write programs for people, not computers

- A program should not require its readers to hold more than a handful of facts in memory at once.
- Make names consistent, distinctive, and meaningful.
- Make code style and formatting consistent.

Let the computer do the work

- Make the computer repeat tasks.
- Save recent commands in a file for re-use.
- Use a build tool to automate workflows.
- Use an Integrated Developing Environment (IDE)

Make incremental changes

- Work in small steps with frequent feedback and course correction.
- Use a version control system.
- Put everything that has been created manually in version control.

Don't repeat yourself (or others)

- Every piece of data must have a single authoritative representation in the system.
- Modularize code rather than copying and pasting.
- Re-use code instead of rewriting it.

Plan for mistakes

- Add assertions to programs to check their operation.
- Use an off-the-shelf unit testing library.
- Turn bugs into test cases.
- Use a symbolic debugger.

Optimize software only after it works correctly

- Use a profiler to identify bottlenecks.
- Write code in the highest-level language possible.

Document design and purpose, not mechanics

- Document interfaces and reasons, not implementations.
- Refactor code in preference to explaining how it works.
- Embed the documentation for a piece of software in that software.

Collaborate

- Use pre-merge code reviews.
- Use pair programming when bringing someone new up to speed and when tackling particularly tricky problems.
- Use an issue tracking tool.